

## LAB 3: LINEAR REGRESSION IMPLEMENTATION

### OBJECTIVES:

1. Apply linear regression to predict house prices.
2. Plot actual vs predicted.

### THEORY:

In linear regression, the output is modeled as a linear combination of input variables:

$$y = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

where  $x_1, x_2, \dots, x_n$  are the input features,  $w_1, w_2, \dots, w_n$  are the corresponding weights, and  $b$  is the bias term. The goal is to find the weights and bias that minimize the difference between predicted outputs and actual targets. The model's performance is commonly evaluated using metrics such as:

Mean Squared Error (MSE):  $MSE = (1/N) * \sum((y_i - y_{pred\_i})^2)$

where  $y_i$  is the true value,  $y_{pred\_i}$  is the predicted value, and  $N$  is the number of samples. Lower MSE indicates that the model's predictions are closer to the actual values, representing better performance. This approach provides a foundation for understanding how linear models can approximate relationships between inputs and outputs.

**TOOLS:** Scikit-learn, NumPy

**DATASET:** Boston Housing

### ALGORITHM:

1. Start
2. Load the Boston Housing dataset features ( $x$ ) and target ( $y$ ).
3. Add a column of ones to  $x$  to represent the bias term ( $b$ ):  $X = [1, x_1, x_2, \dots]$ .
4. Split the augmented dataset into training and testing sets.
5. Solve for the weight vector ( $\theta$ ) using the Normal Equation:  $\theta = \text{inverse}(X_{\text{train\_transpose}} * X_{\text{train}}) * X_{\text{train\_transpose}} * y_{\text{train}}$
6. Predict the house prices for the test set:  $y_{\text{pred}} = X_{\text{test}} * \theta$ .
7. Calculate the Mean Squared Error (MSE).
8. Plot the actual prices versus the predicted prices.
9. End

## SOURCE CODE:

```
# Cell 1: Data Preparation
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split

# Fetch Boston Housing dataset
print("Loading Boston Housing data...")
boston = fetch_openml(data_id=531, as_frame=False, parser='pandas')

x = np.array(boston.data, dtype=float)
y = np.array(boston.target, dtype=float)

# Add bias term (column of ones) to X
ones = np.ones((x.shape[0], 1))
X = np.hstack((ones, x))

# Split data into 80% training and 20% testing
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Cell 2: Implementation of Normal Equation (Analytical Solution)
#  $\theta = (X^T X)^{-1} X^T y$ 
XT = X_train.T
theta = np.linalg.inv(XT @ X_train) @ XT @ y_train

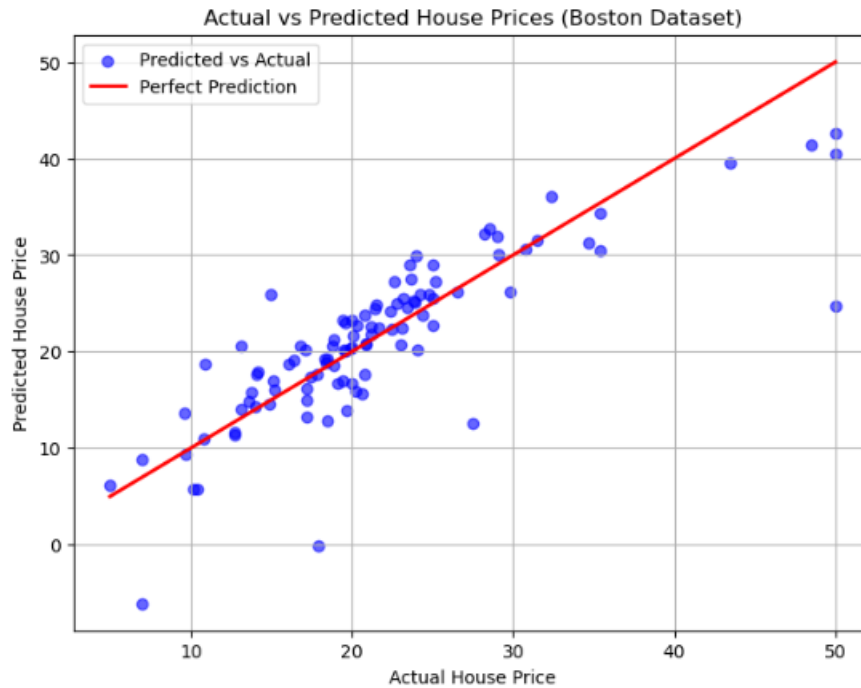
# Cell 3: Prediction and Evaluation
y_pred = X_test @ theta

# Compute MSE manually
mse = np.mean((y_test - y_pred) ** 2)
print(f"Mean Squared Error: {mse:.4f}")

# Cell 4: Visualization
plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_pred, color='blue', alpha=0.6, label='Predicted vs Actual')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], color='red', lw=2, label='Perfect Prediction')
plt.xlabel("Actual House Price")
plt.ylabel("Predicted House Price")
plt.title("Actual vs Predicted House Prices (Boston Dataset)")
plt.legend()
plt.grid(True)
plt.show()
```

## OUTPUTS:

Mean Squared Error: 24.2911



## CONCLUSION:

In this lab, we implemented a Linear Prediction model using the Boston Housing dataset. By utilizing the Normal Equation, we calculated the optimal weights and bias term to minimize the error between predictions and actual values. The performance was evaluated using the Mean Squared Error metric, which yielded a value of 24.29. The final plot shows a strong linear correlation between actual and predicted prices, demonstrating that the linear combination of input features is an effective model for this regression task.